Chat UI (WhatsApp / YouTube live chat)

- **Problem:** Build a chat window like WhatsApp or YouTube live chat. Messages from other people stream in live, the user can send messages, the list sticks to the bottom while the user is reading the latest, and it stays fast when hundreds of messages arrive.
- **Tech stack:** React (Vite, JavaScript, Tailwind CSS v4)
- **Time limit:** 60 minutes
- **Often asked at:** Meta, Microsoft, Swiggy
- **Why it matters:** Auto-scroll, capped message list, polling or websockets

---

PART A: Thought process

General steps for any machine coding task

1. **Understand the requirements.** Restate the problem, split functional vs non-functional, ask clarifying questions, write down assumptions.
2. **List the interactions.** Every user action and its expected result. This is your checklist for building and testing.
3. **Design the architecture.** Name the components and their one-line jobs. Start small.
4. **Design the state.** What is the single source of truth? What can be derived?
5. **Design data structures and contracts.** Shape of the data, the transport (socket / API) contract, hook inputs and outputs.
6. **Trace one data flow end to end.** Proves the design works before you type.
7. **Find the hard parts.** Solve them on paper first.
8. **Build in layers.** Simplest working version first, then one working layer at a time.
9. **Cover states and edge cases.** Connecting, empty, sending, failed, offline, flood of messages.
10. **Wrap up.** Trade-offs and what you would add with more time.

Time budget for 60 minutes

Phase	Time
Requirements + clarifying questions	5 min
Interactions, architecture, state design	7 min
Hard parts on paper (auto-scroll, cap, optimistic send)	5 min
Build: reducer + mock socket + static list	12 min
Build: composer + optimistic send / retry	10 min
Build: auto-scroll + "new messages" pill	10 min
Build: cap + scroll anchor, offline state	6 min
Edge cases, accessibility, wrap-up	5 min

Tip: think out loud

Say what you are deciding and why: "I will only auto-scroll if the user was already at the bottom. If they scrolled up to read, yanking them down is the worst bug a chat can have, so I'll show a '3 new messages' button instead." The interviewer grades your reasoning as much as the code.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- **Live messages:** messages from other users arrive in real time and appear at the bottom.
- **Send:** the user types and presses Enter (or Send). Shift+Enter adds a new line. Empty / whitespace-only messages are not sent.
- **Optimistic send:** my message shows at once with "Sending…", then "✓" when the server acks it, or "Not sent. Retry" if it fails.
- **Auto-scroll:** if the user is at the bottom, new messages keep the view at the bottom. If the user scrolled up, the view stays where it is and a **"↓ N new messages"** button appears. Clicking it scrolls to the latest. Sending a message always jumps to the bottom.
- **Capped list:** keep only the newest 200 messages (live chat can get thousands per minute).
- **Connection status:** Connecting / Online / Offline in the header; an offline banner.
- **Bubbles:** mine on the right, others on the left with the sender's name (shown once per run of messages from the same person), and a time.

Non-functional requirements

Area	What it means here
Performance	A new message re-renders one row, not the whole list. The DOM never grows past 200 rows. No scroll jank.
Scalability	Works with a flood (10 messages per second). The transport can be swapped (WebSocket, polling, SSE) without touching the UI.
Error handling	Failed sends are visible and retryable. Duplicate deliveries (server echo, retries) never show twice.
Accessibility	The list is `role="log"` so screen readers announce new messages. The textarea has a label. Status changes are announced. Buttons are real `<button>`s.
Reusability	`useChat` (data) and `useAutoScroll` (scroll behaviour) are separate hooks; `useAutoScroll` works for any growing list (logs, feeds).
Extensibility	Easy to add typing indicators, read receipts, history loading, multiple rooms.

Clarifying questions to ask the interviewer

1. **Real backend or mock?** Is there a WebSocket server, or should I mock the transport? (Mock it, with the same shape as a real socket.)

2. **One room or a conversation list?** WhatsApp has a sidebar of chats; live chat is one room. (One room.)
3. **History?** Do we load older messages when scrolling up, or only show live ones? (Live only; history is a "more time" item.)
4. **How many messages can arrive?** A few per minute (WhatsApp) or hundreds (YouTube live)? (Up to ~10/s, so we cap the list.)
5. **What happens on failure?** Auto-retry, or a manual Retry button? Queue while offline? (Manual Retry.)
6. **Rich content?** Images, emoji picker, reactions, edits? (Plain text only, emoji typed as text.)

Assumptions we will build with

- One chat room, plain text, max 500 characters.
- A mock socket pushes messages from 3 fake users every 1.5 s (or every 100 ms in "Flood" mode) and acks my messages after 300-1000 ms, failing 15% of the time.
- The server echoes my message back to every client, me included (like a real broadcast).
- Keep the newest 200 messages. No history loading.
- The current user is `{ id: 'me', name: 'You' }`.

---

2. Identify interactions / use cases

Action / input	Expected behavior / output
App opens	Header shows "Connecting…", then "Online". Empty state "No messages yet".
Message from another user arrives, user at bottom	Message appears at the bottom; the view stays pinned to the bottom.
Message arrives, user scrolled up	View does not move. Button "↓ 1 new message" appears; the count grows with each message.
Click "↓ N new messages"	Smooth scroll to the bottom; the button disappears.
User scrolls back to the bottom by hand	The button disappears; auto-scroll resumes.
Type text + Enter	Message appears on the right with "Sending…", input clears, view jumps to the bottom.
Shift+Enter	New line inside the textarea; nothing is sent.
Enter with empty / spaces only	Nothing happens; Send button is disabled.
Server acks	"Sending…" becomes "✓"; time becomes the server time.
Server echoes my message back	Ignored (same id): no duplicate.
Send fails	Bubble gets a red ring and "Not sent. Retry".
Click Retry	Back to "Sending…", same id, then "✓" or failed again.
Click "Go offline"	Status "Offline", red banner, no incoming messages, sends fail.
Click "Go online"	Status "Online", messages flow again; failed ones can be retried.
Click "Flood"	10 messages per second; list never exceeds 200 rows; scrolling stays smooth.

Action / input	Expected behavior / output
Scrolled up during a flood	Reading position stays still even though old messages are removed above it.
Typing with an IME (Japanese, Hindi) and pressing Enter to pick a word	The word is committed; the message is **not** sent.

---

3. Design the architecture

Module	Responsibility
`utils/chat.js`	Pure logic: `chatReducer`, `countUnread`, `isNearBottom`, `formatTime`, constants. No React, easy to test.
`services/mockSocket.js`	Fake server: pushes messages, acks / fails my sends, online / offline, speed. Same shape a real socket wrapper would have.
`hooks/useChat.js`	Owns the messages and connection status. Connects to the socket, exposes `send`, `retry`, `setOnline`, `setSpeed`.
`hooks/useAutoScroll.js`	Owns scroll behaviour: stick to bottom, unread count, holding position when old messages are trimmed.
`components/MessageList.jsx`	Scroll container, renders rows, empty state, "new messages" button.
`components/MessageItem.jsx`	One bubble (memoized): name, text, time, status, Retry.
`components/Composer.jsx`	Textarea + Send. Enter / Shift+Enter, trimming, max length.
`App.jsx`	Layout: header with status and demo controls, offline banner, list, composer.

```
App
├── useChat()  ──────────── mockSocket (server)
├── Header (status, Flood, Go offline)
├── MessageList  ── useAutoScroll(messages)
│   └── MessageItem × N   (memo)
└── Composer  ── onSend → useChat.send
```

Why split this way: the **data** (what messages exist) and the **scroll behaviour** (where the user is looking) change for different reasons, so they live in different hooks. The socket sits behind a tiny interface so swapping mock → WebSocket → polling touches one file.

---

4. Design the state / data model

State or field	Purpose
`messages` (in `useChat`, via `useReducer`)	The single list of messages, oldest first, capped at 200. Includes my pending / failed ones.

State or field	Purpose
`message.status`	Only on my messages: `'sending' | 'sent' | 'failed'`. Incoming ones have none.
`status` (in `useChat`)	Connection: `'connecting' | 'connected' | 'offline'`.
`socketRef` (ref)	The socket instance. A ref, not state: changing it must not re-render.
`lastSeenId` (in `useAutoScroll`)	`null` when the user is at the bottom. Otherwise the id of the last message they had seen when they scrolled away.
`stickRef` (ref)	Was the user at the bottom before the latest update? Read during layout, so a ref.
`anchorRef` (ref)	First visible message and its offset, used to hold the reading position.
`text` (in `Composer`)	The draft. Local: nobody else needs it.
`flood` (in `App`)	Demo toggle for the speed button label.

```
// useChat
const [messages, dispatch] = useReducer(chatReducer, []);
const [status, setStatus] = useState('connecting');
const socketRef = useRef(null);

// useAutoScroll
const stickRef = useRef(true);
const anchorRef = useRef(null);
const [lastSeenId, setLastSeenId] = useState(null);
```

Senior note: derive, don't store.

- **Unread count is derived**, not a counter: `countUnread(messages, lastSeenId)` = messages after `lastSeenId`. A counter (`unread + 1` on every message) breaks when React batches several messages into one render, and never stays right when the cap trims the list.
- **"Show the sender name"** is derived in render: `messages[i - 1]?.user.id !== m.user.id`. Not stored on the message.
- **"At bottom" for the button** is derived: `lastSeenId == null`.
- **Ownership:** `useChat` owns the messages and the socket. `useAutoScroll` owns everything about scroll. The composer owns its draft. `App` only wires them.

---

5. Design the data structures and contracts

Message

```
{
  id: '7b0c…',                       // stable, unique: React key + dedupe + retry idempotency
  user: { id: 'u1', name: 'Asha' },  // who sent it
  text: 'Did the deploy go out?',
```

```
  createdAt: 1762076400000,          // ms timestamp; server time once acked
  status: 'sent',                    // only on my messages: 'sending' | 'sent' | 'failed'
}
```

Why a client-generated id (`crypto.randomUUID()`) for my messages? The optimistic message needs a key before the server answers. Using the same id for the server copy means: the ack can find and update it, the echo can be dropped as a duplicate, and a retry can be dropped by the server if the first attempt actually arrived (idempotency key). Never use the array index as key: the cap shifts every index.

Socket contract (mock now, real later)

```
const socket = createMockSocket({
  onMessage: (message) => {},   // every message from the server (others' + echo of mine)
  onStatus: (status) => {},     // 'connecting' | 'connected' | 'offline'
});

await socket.send({ id, user, text }); // resolves { id, user, text, createdAt } (ack) or throws
socket.setOnline(false);               // demo only
socket.setSpeed(100);                  // demo only: ms between incoming messages
socket.close();                        // stop timers (unmount)
```

The same contract over a real WebSocket:

```
// client -> server
{ "type": "send", "id": "7b0c…", "text": "hi" }
// server -> client (ack, only to me)
{ "type": "ack", "id": "7b0c…", "createdAt": 1762076400000 }
// server -> client (broadcast, to everyone)
{ "type": "message", "message": { "id": "7b0c…", "user": {…}, "text": "hi", "createdAt": … } }
```

Hook contracts

```
const { messages, status, send, retry, setOnline, setSpeed } = useChat();
const { listRef, onScroll, atBottom, unread, scrollToBottom } = useAutoScroll(messages);
```

---

6. Trace the data flow

I send "hi" while scrolled up reading old messages.

```
Enter in textarea
  ↓ Composer.submit: trim → "hi" (not empty) → onSend("hi"), clear draft
  ↓ useChat.send: id = randomUUID()
  ↓ dispatch({ type: 'sending', message })  → reducer appends { …, status: 'sending' }, caps to 200
  ↓ render: new MessageItem on the right "Sending…" (other rows skipped by memo)
  ↓ useAutoScroll layout effect (lastId changed): last message is mine → scrollTop = scrollHeight
```

```
  ↓ browser fires scroll → onScroll: near bottom → stickRef = true, lastSeenId = null → button hides
  ↓ deliver(): await socket.send(message)   (300-1000 ms)
  ├─ ok   → dispatch({ type: 'sent', id, createdAt }) → bubble shows ✓ and server time
  │        → 50 ms later the echo arrives → 'received' with same id → reducer returns same array (no
render)
  └─ fail → dispatch({ type: 'failed', id }) → red ring + "Not sent. Retry"
```

---

7. Important technical challenges

Challenge 1: Auto-scroll only when the user is at the bottom

Problem. The naive version scrolls to the bottom on every new message. Scenario: the user scrolls up to re-read a question from 2 minutes ago; a message arrives every second and throws them back to the bottom each time. The chat is unusable. The opposite naive version (never scroll) means a user watching live has to scroll by hand after every message.

Solution.

1. On every scroll event, record whether the user is near the bottom (within 60 px, not exactly 0: sub-pixel rounding and zoom make "exactly at bottom" unreliable).
2. When a new last message renders, in `useLayoutEffect` (after the DOM update, before paint, so there is no visible jump): if the user **was** at the bottom, or the message is mine, set `scrollTop = scrollHeight`. Otherwise leave it.
3. The check must use the state from **before** the new message was added. After adding it, the user is no longer near the bottom by definition. That is why it is stored in a ref during `onScroll`.
4. Depend on the last message id, not the whole array: an ack (`sending → sent`) must not scroll anything.

```
export const isNearBottom = (el, threshold = 60) =>
  el.scrollHeight - el.scrollTop - el.clientHeight < threshold;

const onScroll = () => {
  const atBottom = isNearBottom(listRef.current);
  stickRef.current = atBottom;
  setLastSeenId((prev) => (atBottom ? null : (prev ?? lastId)));
};

useLayoutEffect(() => {
  const el = listRef.current;
  const mine = messages.at(-1)?.user.id === ME.id;
  if (stickRef.current || mine) el.scrollTop = el.scrollHeight;
}, [lastId]);
```

Challenge 2: "N new messages" count that stays correct

Problem. A counter `setUnread(n => n + 1)` inside an effect undercounts: in a flood, React batches 3 socket messages into one render, so the effect runs once and the counter says 1 instead of 3. It also needs a reset in several places, and it cannot know about messages removed by the cap.

Solution. Store **which message was last seen** (snapshot `lastId` when the user leaves the bottom) and **derive** the count: the number of messages after it. If it was trimmed away by the cap, every message in the list is new, so show "200+".

```
export function countUnread(messages, lastSeenId) {
  if (lastSeenId == null) return 0;
  const i = messages.findIndex((m) => m.id === lastSeenId);
  return i === -1 ? messages.length : messages.length - 1 - i; // -1: trimmed by the cap
}
```

Challenge 3: Capping the list without the content jumping

Problem. Live chat can push 10 messages per second. Keeping all of them means 36 000 DOM nodes after an hour and a slow, janky page. So we keep the newest 200 (`slice(-200)`). But removing messages from the **top** while the user is reading in the middle shifts everything up: scenario, the user is reading message #120, 5 old messages are removed above it, and the text they were reading jumps ~300 px up under their eyes.

Solution: scroll anchoring by hand. On every scroll, remember the first visible message and its offset from the top of the viewport. After each update (in the layout effect, before paint), find that same message again and set `scrollTop` so it sits at the same offset. Browsers have a built-in version (`overflow-anchor`), but it is not supported everywhere (Safari) and is easy to fight with, so we turn it off with `overflow-anchor: none` and do it ourselves. The list needs `position: relative` so `offsetTop` is measured from it.

```
const captureAnchor = () => {
  const el = listRef.current;
  const first = [...el.children].find((c) => c.offsetTop + c.offsetHeight > el.scrollTop);
  anchorRef.current = first ? { id: first.dataset.id, top: first.offsetTop - el.scrollTop } : null;
};

// in the layout effect, when the user is NOT at the bottom:
const a = anchorRef.current;
const node = a && el.querySelector(`[data-id="${a.id}"]`);
if (node) el.scrollTop = node.offsetTop - a.top;
```

The reducer keeps object references for unchanged messages, and `MessageItem` is wrapped in `memo`, so each new message renders one row even when 200 are on screen.

Challenge 4: Optimistic send, acks, echoes and retries without duplicates

Problem. Showing my message only after the server answers feels laggy (300 ms+). Showing it at once raises three questions: how does the ack find it, what happens when the server broadcasts my own message back to me (scenario: the message appears twice), and what if a "failed" send actually reached the server and I retry (scenario: the other side sees it twice)?

Solution. One client-generated id is used everywhere. The reducer appends with `status: 'sending'`; the ack patches by id; the echo is dropped because the id already exists; retry re-sends with the **same** id so the server can ignore it as a duplicate (idempotency).

```
case 'received':
  if (messages.some((m) => m.id === action.message.id)) return messages; // echo / duplicate
  return cap([...messages, action.message]);
case 'sending':
  return cap([...messages, { ...action.message, status: 'sending' }]);
case 'sent':
  return patch(messages, action.id, { status: 'sent', createdAt: action.createdAt });
case 'failed':
  return patch(messages, action.id, { status: 'failed' });
```

---

8. Implement in the correct order

#	Step	Working result
1	Layout: header, empty scroll area, textarea + button	Static chat window
2	`chatReducer` with `received` + `MessageItem` with hard-coded messages	Bubbles render, mine on the right
3	`mockSocket` + `useChat` connecting, pushing messages, status	Live messages appear; "Connecting…" → "Online"
4	`Composer`: Enter / Shift+Enter, trim, disabled Send	Can type; nothing sent yet
5	Optimistic `send` + `sent` / `failed` actions + ack in the socket	My messages show Sending… → ✓
6	Retry button + `retry` action; echo dedupe	Failed messages recover; no duplicates
7	Auto-scroll: `isNearBottom` + `stickRef` + layout effect	Stays at the bottom; doesn't yank when scrolled up
8	`lastSeenId` + `countUnread` + "↓ N new messages" button	User knows new messages arrived
9	Cap at 200 + `memo` on `MessageItem` + Flood button	Stays fast under load
10	Scroll anchor for trimmed messages	Reading position holds during a flood
11	Offline toggle + banner, sender-name grouping, a11y (`role="log"`, label)	Polished, accessible

---

9. Cover all states and edge cases

State / edge case	What the UI does
Connecting (initial)	Amber dot "Connecting…"; the offline toggle is disabled.
Connected, no messages (empty)	"No messages yet. Say hi 👋" centred in the list.
Connected, messages	Bubbles; view pinned to the bottom.
Sending (processing)	My bubble with "Sending…".
Sent (success)	"✓" and server time.

State / edge case	What the UI does
Failed (error)	Red ring + "Not sent. Retry".
Offline	Red dot, red banner (`role="alert"`), incoming stops, sends fail.
Scrolled up + new messages	View holds; "↓ N new messages" button.
Scrolled up, no new messages	"↓ Jump to latest" button.
Unread message trimmed by the cap	Button shows "200+ new messages".
Flood (10/s)	Max 200 rows; one row renders per message; anchor keeps position.
Empty / whitespace message	Not sent; Send disabled.
Very long word / URL	`break-words` wraps it inside the bubble.
Multi-line message	`whitespace-pre-wrap` keeps line breaks.
Message over 500 chars	`maxLength` stops typing (the server must check too).
HTML / script in a message	Rendered as text: React escapes it. Never `dangerouslySetInnerHTML` .
IME composition + Enter	Not sent (`isComposing` check).
Duplicate delivery / echo	Dropped by id in the reducer.
Ack after unmount	Socket closed in effect cleanup; React ignores the late dispatch.
React StrictMode double effect	First socket closed by cleanup, second one used: no double messages.
Same sender several times in a row	Name shown once for the run.

10. Final structure

```
_71_chat_ui/
├── index.html
├── package.json
├── vite.config.js
└── src/
    ├── main.jsx
    ├── index.css
    ├── App.jsx                    # layout, header, offline banner
    ├── utils/
    │   └── chat.js                # reducer, countUnread, isNearBottom, formatTime
    ├── services/
    │   └── mockSocket.js          # fake server (swap for WebSocket)
    ├── hooks/
    │   ├── useChat.js             # messages + connection + send/retry
    │   └── useAutoScroll.js       # stick to bottom, unread, anchor
    └── components/
        ├── MessageList.jsx        # scroll container + new-messages button
```

```
        ├── MessageItem.jsx         # one bubble (memo)
        └── Composer.jsx            # textarea + send
```

---

11. Full working code

Create the project with `npm create vite@latest` (React, JavaScript), then `npm i -D tailwindcss @tailwindcss/vite`.

`vite.config.js`

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react-swc';
import tailwindcss from '@tailwindcss/vite';

export default defineConfig({
  plugins: [react(), tailwindcss()],
});
```

`index.html`

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Chat UI</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

`src/index.css`

```
@import "tailwindcss";
```

`src/main.jsx`

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import './index.css';
import App from './App';
```

```
createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

src/utils/chat.js

```
export const ME = { id: 'me', name: 'You' };
export const MAX_MESSAGES = 200; // keep the DOM small on a busy live chat
export const MAX_LENGTH = 500;

const cap = (list) => (list.length > MAX_MESSAGES ? list.slice(-MAX_MESSAGES) : list);

// Replace only the changed message so every other message keeps its reference (memo works).
const patch = (messages, id, changes) =>
  messages.map((m) => (m.id === id ? { ...m, ...changes } : m));

export function chatReducer(messages, action) {
  switch (action.type) {
    case 'received':
      // The server also echoes my own messages back: same id, so skip duplicates.
      if (messages.some((m) => m.id === action.message.id)) return messages;
      return cap([...messages, action.message]);
    case 'sending':
      return cap([...messages, { ...action.message, status: 'sending' }]);
    case 'sent':
      return patch(messages, action.id, { status: 'sent', createdAt: action.createdAt });
    case 'failed':
      return patch(messages, action.id, { status: 'failed' });
    case 'retry':
      return patch(messages, action.id, { status: 'sending' });
    default:
      return messages;
  }
}

// How many messages arrived after the last one the user saw at the bottom.
export function countUnread(messages, lastSeenId) {
  if (lastSeenId == null) return 0;
  const i = messages.findIndex((m) => m.id === lastSeenId);
  return i === -1 ? messages.length : messages.length - 1 - i; // -1: trimmed by the cap
}

export const isNearBottom = (el, threshold = 60) =>
  el.scrollHeight - el.scrollTop - el.clientHeight < threshold;
```

```
export const formatTime = (ts) =>
  new Date(ts).toLocaleTimeString([], { hour: '2-digit', minute: '2-digit' });
```

src/services/mockSocket.js

```
const USERS = [
  { id: 'u1', name: 'Asha' },
  { id: 'u2', name: 'Rahul' },
  { id: 'u3', name: 'Meera' },
];
const LINES = [
  'Hey everyone 👋',
  'Did the deploy go out?',
  'lol',
  'Looks good to me',
  'Can someone review my PR?',
  'brb, coffee',
  'This stream is 🔥',
  'Which timezone is the meeting in?',
  'A longer message to check that bubbles wrap nicely when people write a whole paragraph in chat.',
];
const pick = (list) => list[Math.floor(Math.random() * list.length)];
const wait = (ms) => new Promise((resolve) => setTimeout(resolve, ms));

// Fake server: pushes messages from other users and acks mine (WebSocket + socket.io-style ack).
export function createMockSocket({ onMessage, onStatus, intervalMs = 1500, failRate = 0.15 }) {
  let online = true;
  let closed = false;
  let timer;

  const schedule = () => {
    clearInterval(timer);
    timer = setInterval(() => {
      if (online) onMessage({ id: crypto.randomUUID(), user: pick(USERS), text: pick(LINES), createdAt:
Date.now() });
    }, intervalMs);
  };

  onStatus('connecting');
  const connectTimer = setTimeout(() => {
    onStatus('connected');
    schedule();
  }, 500);

  return {
    async send({ id, user, text }) {
      await wait(300 + Math.random() * 700);
```

```
      if (!online || Math.random() < failRate) throw new Error('Network error');
      const saved = { id, user, text, createdAt: Date.now() };
      setTimeout(() => !closed && onMessage(saved), 50); // broadcast echo to every client, me included
      return saved;
    },
    setOnline(next) {
      online = next;
      onStatus(next ? 'connected' : 'offline');
    },
    setSpeed(ms) {
      intervalMs = ms;
      schedule();
    },
    close() {
      closed = true;
      clearTimeout(connectTimer);
      clearInterval(timer);
    },
  };
}
```

src/hooks/useChat.js

```
import { useCallback, useEffect, useReducer, useRef, useState } from 'react';
import { chatReducer, ME } from '../utils/chat';
import { createMockSocket } from '../services/mockSocket';

export function useChat() {
  const [messages, dispatch] = useReducer(chatReducer, []);
  const [status, setStatus] = useState('connecting');
  const socketRef = useRef(null);

  useEffect(() => {
    const socket = createMockSocket({
      onMessage: (message) => dispatch({ type: 'received', message }),
      onStatus: setStatus,
    });
    socketRef.current = socket;
    return () => socket.close();
  }, []);

  const deliver = useCallback(async (message) => {
    try {
      const saved = await socketRef.current.send(message);
      dispatch({ type: 'sent', id: message.id, createdAt: saved.createdAt });
    } catch {
      dispatch({ type: 'failed', id: message.id });
```

```
    }
  }, []);

  // Optimistic: show the message at once, then confirm or mark it failed.
  const send = useCallback(
    (text) => {
      const message = { id: crypto.randomUUID(), user: ME, text, createdAt: Date.now() };
      dispatch({ type: 'sending', message });
      deliver(message);
    },
    [deliver]
  );

  // Same id on retry, so the server can drop it if the first attempt actually got through.
  const retry = useCallback(
    (message) => {
      dispatch({ type: 'retry', id: message.id });
      deliver(message);
    },
    [deliver]
  );

  const setOnline = useCallback((online) => socketRef.current?.setOnline(online), []);
  const setSpeed = useCallback((ms) => socketRef.current?.setSpeed(ms), []);

  return { messages, status, send, retry, setOnline, setSpeed };
}
```

src/hooks/useAutoScroll.js

```
import { useLayoutEffect, useRef, useState } from 'react';
import { countUnread, isNearBottom, ME } from '../utils/chat';

export function useAutoScroll(messages) {
  const listRef = useRef(null);
  const stickRef = useRef(true); // was the user at the bottom before the latest message came in?
  const anchorRef = useRef(null); // first visible message + its offset, to hold the reading position
  const [lastSeenId, setLastSeenId] = useState(null); // null = at the bottom
  const lastId = messages.at(-1)?.id;

  const captureAnchor = () => {
    const el = listRef.current;
    const first = [...el.children].find((c) => c.offsetTop + c.offsetHeight > el.scrollTop);
    anchorRef.current = first ? { id: first.dataset.id, top: first.offsetTop - el.scrollTop } : null;
  };

  const onScroll = () => {
```

```
    const atBottom = isNearBottom(listRef.current);
    stickRef.current = atBottom;
    setLastSeenId((prev) => (atBottom ? null : (prev ?? lastId)));
    captureAnchor();
  };

  // Runs after the new message is in the DOM but before paint: no visible jump.
  useLayoutEffect(() => {
    const el = listRef.current;
    const mine = messages.at(-1)?.user.id === ME.id;
    if (stickRef.current || mine) {
      el.scrollTop = el.scrollHeight;
    } else {
      // Scrolled up: the cap may have removed messages above, so put the anchor back where it was.
      const a = anchorRef.current;
      const node = a && el.querySelector(`[data-id="${a.id}"]`);
      if (node) el.scrollTop = node.offsetTop - a.top;
    }
    captureAnchor();
    // eslint-disable-next-line react-hooks/exhaustive-deps -- only a new last message should scroll,
not status updates
  }, [lastId]);

  const scrollToBottom = () =>
    listRef.current.scrollTo({ top: listRef.current.scrollHeight, behavior: 'smooth' });

  return {
    listRef,
    onScroll,
    atBottom: lastSeenId == null,
    unread: countUnread(messages, lastSeenId),
    scrollToBottom,
  };
}
```

src/components/MessageItem.jsx

```
import { memo } from 'react';
import { formatTime, ME } from '../utils/chat';

const STATUS_TEXT = { sending: 'Sending…', sent: '✓' };

// memo: a new message re-renders only itself, not the other 199.
const MessageItem = memo(function MessageItem({ message, showName, onRetry }) {
  const mine = message.user.id === ME.id;

  return (
```

```
    <li data-id={message.id} className={`flex flex-col px-3 ${mine ? 'items-end' : 'items-start'}
${showName ? 'pt-3' : 'pt-1'}`}>
      {showName && !mine && <span className="mb-0.5 text-xs font-semibold text-teal-700">
{message.user.name}</span>}
      <div
        className={`max-w-[80%] rounded-2xl px-3 py-1.5 text-sm whitespace-pre-wrap break-words shadow-
sm ${
          mine ? 'rounded-br-sm bg-emerald-100' : 'rounded-bl-sm bg-white'
        } ${message.status === 'failed' ? 'ring-1 ring-red-400' : ''}`}
      >
        {message.text}
        <span className="ml-2 align-bottom text-[10px] text-gray-500">
          {formatTime(message.createdAt)} {mine && STATUS_TEXT[message.status]}
        </span>
      </div>
      {message.status === 'failed' && (
        <button type="button" onClick={() => onRetry(message)} className="mt-0.5 text-xs text-red-600
hover:underline">
          Not sent. Retry
        </button>
      )}
    </li>
  );
});

export default MessageItem;
```

src/components/MessageList.jsx

```
import { useAutoScroll } from '../hooks/useAutoScroll';
import { MAX_MESSAGES } from '../utils/chat';
import MessageItem from './MessageItem';

export default function MessageList({ messages, onRetry }) {
  const { listRef, onScroll, atBottom, unread, scrollToBottom } = useAutoScroll(messages);

  return (
    <div className="relative min-h-0 flex-1">
      <ol
        ref={listRef}
        onScroll={onScroll}
        role="log"
        aria-label="Messages"
        className="relative h-full overflow-y-auto pb-3 [overflow-anchor:none]"
      >
        {messages.map((m, i) => (
          <MessageItem key={m.id} message={m} showName={messages[i - 1]?.user.id !== m.user.id}
```

```
onRetry={onRetry} />
        ))}
      </ol>

      {messages.length === 0 && (
        <p className="absolute inset-0 grid place-items-center text-sm text-gray-500">No messages yet. Say hi 👋</p>
      )}

      {!atBottom && (
        <button
          type="button"
          onClick={scrollToBottom}
          className="absolute bottom-3 left-1/2 -translate-x-1/2 rounded-full bg-teal-700 px-4 py-1.5 text-sm text-white shadow-lg"
        >
          ↓ {unread > 0 ? `${unread >= MAX_MESSAGES ? `${MAX_MESSAGES}+` : unread} new message${unread === 1 ? '' : 's'}` : 'Jump to latest'}
        </button>
      )}
    </div>
  );
}
```

src/components/Composer.jsx

```
import { useState } from 'react';
import { MAX_LENGTH } from '../utils/chat';

export default function Composer({ onSend }) {
  const [text, setText] = useState('');

  const submit = (e) => {
    e.preventDefault();
    const value = text.trim();
    if (!value) return;
    onSend(value);
    setText('');
  };

  const onKeyDown = (e) => {
    // Enter sends, Shift+Enter is a new line; isComposing: don't send mid-word in IME input (Hindi, Japanese)
    if (e.key === 'Enter' && !e.shiftKey && !e.nativeEvent.isComposing) submit(e);
  };

  return (
```

```
    <form onSubmit={submit} className="flex items-end gap-2 border-t border-gray-200 bg-white p-2">
      <label htmlFor="composer" className="sr-only">
        Message
      </label>
      <textarea
        id="composer"
        rows={1}
        maxLength={MAX_LENGTH}
        value={text}
        onChange={(e) => setText(e.target.value)}
        onKeyDown={onKeyDown}
        placeholder="Type a message"
        className="max-h-32 flex-1 resize-none rounded-2xl border border-gray-300 px-3 py-2 text-sm
field-sizing-content focus:border-teal-600 focus:outline-none"
      />
      <button
        type="submit"
        disabled={!text.trim()}
        className="rounded-full bg-teal-700 px-4 py-2 text-sm font-medium text-white disabled:opacity-
40"
      >
        Send
      </button>
    </form>
  );
}
```

src/App.jsx

```
import { useState } from 'react';
import { useChat } from './hooks/useChat';
import MessageList from './components/MessageList';
import Composer from './components/Composer';

const STATUS = {
  connecting: { label: 'Connecting…', dot: 'bg-amber-400' },
  connected: { label: 'Online', dot: 'bg-green-500' },
  offline: { label: 'Offline', dot: 'bg-red-500' },
};

export default function App() {
  const { messages, status, send, retry, setOnline, setSpeed } = useChat();
  const [flood, setFlood] = useState(false);

  const toggleFlood = () => {
    setSpeed(flood ? 1500 : 100);
    setFlood(!flood);
```

```
  };

  return (
    <div className="mx-auto flex h-dvh max-w-md flex-col bg-stone-100 shadow-xl">
      <header className="flex items-center gap-3 bg-teal-700 px-4 py-3 text-white">
        <div className="flex-1">
          <h1 className="font-semibold">Team chat</h1>
          <p className="flex items-center gap-1.5 text-xs text-teal-100" aria-live="polite">
            <span className={`h-2 w-2 rounded-full ${STATUS[status].dot}`} />
            {STATUS[status].label}
          </p>
        </div>
        <button type="button" onClick={toggleFlood} aria-pressed={flood} className="rounded bg-white/15
px-2 py-1 text-xs">
          {flood ? 'Normal speed' : 'Flood'}
        </button>
        <button
          type="button"
          onClick={() => setOnline(status === 'offline')}
          disabled={status === 'connecting'}
          className="rounded bg-white/15 px-2 py-1 text-xs disabled:opacity-50"
        >
          {status === 'offline' ? 'Go online' : 'Go offline'}
        </button>
      </header>

      {status === 'offline' && (
        <p role="alert" className="bg-red-50 px-4 py-1.5 text-xs text-red-700">
          You are offline. New messages will fail; tap Retry when you are back.
        </p>
      )}

      <MessageList messages={messages} onRetry={retry} />
      <Composer onSend={send} />
    </div>
  );
}
```

---

12. Wrap-up

Edge cases handled

- Auto-scroll only when the user was at the bottom or just sent a message; never on status updates.
- Correct unread count even when React batches messages or the cap trims the last-seen one.
- Reading position holds while old messages are removed above it.
- Optimistic send with ack, failure, retry (same id) and echo dedupe.

- Offline state, connecting state, empty state.
- Empty / whitespace messages, long words, line breaks, max length, IME composition.
- XSS-safe rendering; StrictMode-safe socket lifecycle.

Trade-offs to mention

Decision	Why	Cost
Cap at 200 instead of virtualization	3 lines of code; DOM stays small; matches YouTube live chat	Old messages are gone; can't scroll back far
Manual scroll anchor instead of `overflow-anchor`	Works in every browser, predictable	~10 lines; one `querySelector` per update
`some()` dedupe (O(n))	n ≤ 200, simpler than a Set kept in sync	Would need a `Set` of ids if the cap were 10 000
Manual Retry instead of auto-retry / outbox	User stays in control; no surprise sends	Offline messages need a tap each
Message stays in its original slot after retry	Simple, the user sees what they wrote where they wrote it	Real order on the server may differ
WebSocket-style push (mocked)	Real-time, low latency, no wasted requests	Needs reconnect logic and a stateful server

What to add with more time

- **History:** load older messages on scroll to top (cursor `before=<id>`), keeping the scroll position while prepending (same anchor trick).
- **Virtualization** (`react-window` / TanStack Virtual with dynamic heights) if the full history must stay scrollable.
- **Reconnect with exponential backoff + jitter**, and fetch missed messages with `since=<lastId>` after reconnecting.
- **Outbox:** queue messages while offline and send in order when back online (persist in IndexedDB).
- Typing indicators (throttled), read receipts (✓✓), reactions, date dividers, unread divider line.
- Tests: reducer unit tests (done as a node assert script), React Testing Library for Enter / Shift+Enter, Playwright for scroll behaviour.
- Rate-limit the `aria-live` announcements during a flood (screen readers can't keep up with 10/s).

---

13. Counter questions the interviewer may ask

Requirements

Q1. Why mock the socket instead of just using `setInterval` in the component?

The mock has the same contract as a real transport (`onMessage`, `onStatus`, `send` → ack). The UI never knows it's fake, so replacing it with a WebSocket wrapper is a one-file change. Timers inside components would mix transport logic with rendering and be hard to swap or test.

Q2. WebSocket, long polling, short polling or Server-Sent Events: which one and why?

WebSocket for chat: two-way, low latency, one connection. SSE is great for one-way streams (live comments you only read) and reconnects automatically; sends go over normal POSTs. Short polling (`GET /messages?since=id` every 2-3 s) is the simplest and works through any proxy but wastes requests and adds latency. Long polling is the fallback when WebSockets are blocked. Our `useChat` hides the choice behind one interface.

Q3. How would short polling look with this design?

Same reducer, different transport. Poll with a cursor so you only get new messages, and dedupe handles overlaps:

```
const res = await fetch(`/api/messages?since=${lastId}`);
(await res.json()).forEach((m) => dispatch({ type: 'received', message: m }));
```

Pause polling when the tab is hidden (`document.visibilityState`) and back off on errors.

State design

Q4. Why `useReducer` and not `useState` for messages?

There are five ways the list changes (received, sending, sent, failed, retry). A reducer puts all transitions in one pure, testable function and avoids stale-closure bugs: socket callbacks just `dispatch`, they never read `messages`.

Q5. Why is the unread count derived instead of a counter?

React batches updates: three messages can land in one render, so an effect that does `+1` counts one. Storing "last seen id" and computing "how many after it" is always right, including when the cap trims messages.

Q6. Why are `stickRef` and `anchorRef` refs, not state?

They are read inside the layout effect and written on every scroll event. Nothing renders from them, so state would only cause extra renders on every scroll pixel.

Challenge: auto-scroll

Q7. Why `useLayoutEffect` instead of `useEffect` for scrolling?

`useLayoutEffect` runs after the DOM is updated but before the browser paints. Setting `scrollTop` there means the user never sees a frame where the new message is below the fold and then jumps. With `useEffect` you can get a visible flicker.

Q8. Why a 60 px threshold and not "exactly at the bottom"?

`scrollTop` can be fractional (zoom, high-DPI), and the user may be a few pixels above the end while still "reading the latest". A small threshold makes sticking feel natural.

Q9. What if images load after the message renders and change its height?

The scroll position set at layout time is then too short. Fix: give images fixed dimensions (from the server's metadata) so the height is known up front, or watch the list with a `ResizeObserver` and re-stick to the bottom if `stickRef` is true.

Challenge: capping and anchoring

Q10. What if the user wants to scroll back through 10 000 messages?

A cap doesn't fit then. Keep the full list (or load pages from the server on demand) and virtualize: only render the ~20 rows in view with a library like TanStack Virtual that supports dynamic row heights and reverse (bottom-anchored) lists.

Q11. Why not just use the browser's `overflow-anchor`?

It isn't supported in every browser (Safari has been late), it picks its own anchor node, and it fights with our own `scrollTop` changes. Doing it ourselves is ~10 lines and behaves the same everywhere.

Q12. YouTube pauses live chat when you scroll up. Would you do that instead?

It's a valid alternative: while scrolled up, buffer incoming messages outside the rendered list and flush them when the user returns. Nothing is trimmed under the reader, so no anchor is needed, but the buffer needs its own cap. It's a product decision; mention both.

Challenge: optimistic send

Q13. What if the ack never arrives (server crashed after receiving)?

Add a timeout: if no ack within ~10 s, mark it failed. On retry the same id is sent, so if the first one did land, the server dedupes it and just acks again. That's why the id is created on the client.

Q14. Two messages are sent quickly and the second acks first. Is order a problem?

The UI keeps client order, which is what the sender expects. The server orders by its own time or sequence number, so other people may see a different order. For strict order, send one at a time from an outbox queue, or have the server assign sequence numbers and sort by them.

Q15. How would you handle offline sending like WhatsApp (clock icon, sends later)?

Put messages in an outbox (state + IndexedDB) instead of sending immediately. When status becomes `connected`, flush the outbox in order. Show a clock icon while queued. The same id makes repeated flushes safe.

Performance

Q16. How do you make sure one new message doesn't re-render 200 rows?

The reducer only creates a new object for the message that changed (`map` returns the same reference for the others), and `MessageItem` is wrapped in `memo`. `onRetry` is a stable `useCallback`, and `showName` is a boolean, so memo comparisons pass.

Q17. What if 1 000 messages per second arrive?

Rendering per message is too much. Batch: collect incoming messages in a buffer and flush once per animation frame (or every 100 ms) with one `receivedMany` action. Also throttle the screen-reader announcements and consider sampling (YouTube shows "top chat" for this reason).

Q18. Isn't `messages.some(...)` for dedupe slow?

It's O(n) with n ≤ 200, which is microseconds. If the cap grew to thousands, keep a `Set` of ids next to the array (or a `Map` by id plus an array of ids for order).

Implementation details

Q19. Why check `e.nativeEvent.isComposing` on Enter?

With an IME (Japanese, Chinese, Hindi transliteration), Enter confirms the chosen word. Without the check, the half-typed message would be sent.

Q20. How do you prevent XSS from other users' messages?

Render text as a React child: React escapes it. Never use `dangerouslySetInnerHTML` for messages. If you add links or markdown, parse into React elements or sanitize with DOMPurify, and validate length / content on the server.

Q21. What breaks in StrictMode?

Effects run twice in development: connect, disconnect, connect. Because the effect returns `socket.close()`, the first socket is cleaned up and only one stays alive, so no double messages. Without the cleanup you'd see every message twice.

Trade-offs / "why not X"

Q22. Why not keep messages in Redux / Zustand / Context?

One screen owns them, so a hook is enough. With multiple rooms, a sidebar showing unread counts, or notifications elsewhere in the app, a store keyed by room id would make sense.

Q23. How would you add "user is typing…"?

Send a `typing` event throttled to once every ~2 s while the user types. On the receiving side, store `typingUsers` with an expiry timestamp and hide the indicator if no event arrives for ~3 s. Never put typing state in the message list.